SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO4 – Precision	**Assessment:**	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	35% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	Y SRAVAN KUMAR	**Reg. No.:**	192472289
Section / Batch:	CSE AI 2024	**Date:**	20-08-2026

Code of Conduct

I _Y SRAVAN KUMAR_ (192472289) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____SRAVAN_____

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. AI-Powered Insurance Claim Processing System

An insurance company is developing an NLP-based system to automatically process customer insurance claims. The system uses **Context-Free Grammar (CFG)** to parse customer statements and extract information such as the claimant, damaged object, cause of damage, and claim amount.

However, the system faces the following problems:

- Ambiguous sentence structures
- Difficulty distinguishing noun and prepositional phrase attachment
- Subject–verb agreement errors
- Inefficient parsing of lengthy customer descriptions
- Difficulty handling conversational and incomplete input

Example customer statement:

"I reported the damage to the car after the accident."

The sentence can produce different interpretations depending on whether **"to the car"** is associated with *reported* or *damage*.

Tasks:

a) Analyze the structural ambiguity in the given sentence using CFG and illustrate the possible interpretations.

b) Evaluate the limitations of a basic CFG in handling ambiguity, agreement, and long customer-generated insurance statements.

c) Design an improved parsing approach using **PCFG, Feature Structures, and Earley Parsing** to resolve ambiguity and efficiently process the claim.

d) Justify how the proposed architecture can improve the accuracy, scalability, and reliability of automated insurance claim processing.

---

2. Intelligent Railway Ticket Booking Assistant

A railway company is developing a conversational AI assistant for ticket booking. The system currently uses **top-down parsing** to interpret passenger requests.

The system encounters:

- Excessive backtracking
- Ambiguous attachment of phrases
- Incomplete passenger requests
- Long-distance dependencies
- Slow response for complex queries

Example passenger request:

"Book two tickets from Chennai to Delhi for my parents with AC sleeper."

The system sometimes incorrectly associates **"with AC sleeper"** with the passengers instead of the ticket/class information.

Tasks:

a) Analyze the possible syntactic interpretations of the given passenger request using CFG.

b) Evaluate why top-down parsing may result in excessive backtracking and inefficient processing for conversational ticket-booking queries.

c) Analyze how **Earley Parsing** can handle ambiguity, incomplete input, and different possible parse structures more effectively.

d) Compare **Top-Down Parsing, CFG-based parsing, and Earley Parsing** in terms of ambiguity handling, computational efficiency, partial-input processing, and suitability for real-time railway booking systems.

---

3. AI-Based Legal Contract Analysis System

A legal technology company is developing an NLP system to analyze contracts and automatically identify:

- Parties involved
- Obligations

- Conditions
- Deadlines
- Actions
- Exceptions

The existing system uses a basic CFG parser. However, it produces incorrect interpretations when processing long legal sentences containing relative clauses, passive constructions, and multiple prepositional phrases.

Example contract sentence:

"The supplier who delivered the equipment to the company must replace the defective components within thirty days."

The system sometimes incorrectly identifies the **company** as the entity responsible for delivering the equipment.

Tasks:

a) Analyze the syntactic structure of the given sentence using CFG and identify possible sources of ambiguity.

b) Evaluate why basic CFG is insufficient for accurately interpreting complex legal sentences containing relative clauses and nested structures.

c) Design an NLP architecture integrating **CFG, PCFG, Feature Structures, and Earley Parsing** to improve syntactic and semantic interpretation.

d) Develop a step-by-step processing workflow showing how the system can extract:

- Supplier
- Action
- Equipment
- Company
- Obligation
- Deadline

and justify how the proposed approach improves automated contract analysis.

---

4. AI-Based E-Commerce Product Search System

An e-commerce company is developing a conversational product-search system. Customers enter natural-language queries rather than selecting predefined filters.

The current system uses CFG but faces difficulties with:

- Ambiguous phrase attachment
- Coordination
- Missing words
- Informal customer queries
- Long product descriptions
- Real-time search requirements

Example user query:

"Find laptops with a touchscreen for students under ₹60,000."

The system may incorrectly associate **"for students"** with the touchscreen rather than the laptop.

Tasks:

a) Analyze the syntactic ambiguity in the given query and construct possible parse interpretations using CFG.

b) Evaluate the limitations of CFG in handling informal, elliptical, and ambiguous e-commerce search queries.

c) Propose an improved parsing architecture using **PCFG, Feature Structures, and Earley Parsing** to identify product attributes, user requirements, and price constraints.

d) Justify how the proposed solution can improve search precision, response time, and customer experience in a large-scale e-commerce platform.

---

5. Intelligent Airline Voice Assistant

An airline is developing a real-time voice assistant that allows passengers to modify flight bookings using spoken commands.
The system currently uses a conventional top-down parser. It experiences:

- Backtracking
- Ambiguous attachment
- Speech recognition errors
- Incomplete commands
- Delayed responses
- Difficulty interpreting corrections made by users

Example spoken command:
"Change my flight to Mumbai tomorrow with two passengers."
Depending on the parse, **"with two passengers"** may be incorrectly interpreted as an attribute of the destination rather than the number of passengers.
The passenger may also say:
"Change my flight to Mumbai tomorrow... actually, make that Bangalore."
Tasks:
a) Analyze the possible syntactic structures and ambiguities in the given voice command.
b) Evaluate the limitations of top-down parsing when processing incomplete, corrected, and ambiguous spoken commands in real time.
c) Design a parsing architecture using **Earley Parsing, PCFG, and Feature Structures** to support partial input, ambiguity resolution, and agreement constraints.
d) Develop a processing workflow showing how speech input can be converted into a structured booking request and justify the proposed method for real-time airline applications.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

		performance evaluation			
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1	3	3	3	3	4		16	78
2								
3								
4								
5								

Faculty In-charge	Course Coordinator	Program Director
Signature: Dr.T.Kumaragurubaran_ _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. AI-Powered Insurance Claim Processing System

a) Structural Ambiguity Using CFG

Given sentence:

"I reported the damage to the car after the accident."

A CFG can produce more than one parse because the phrase **"to the car"** can be attached to different parts of the sentence.

A simplified CFG is:

S → NP VP

VP → V NP

VP → V NP PP

NP → Det N

NP → NP PP

PP → P NP

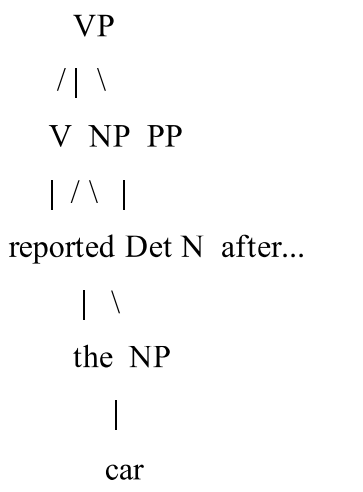
Interpretation 1 – "to the car" modifies damage

I reported [the damage [to the car]] [after the accident].

Meaning:

The car was the object that was damaged, and the damage was reported after the accident.

Structure:



Interpretation 2 – "to the car" is attached to reported

I reported [the damage] [to the car] [after the accident].

Here, "to the car" is interpreted as being associated with the reporting action rather than specifying the damaged object.

Thus, the same words can produce different syntactic structures.

Main issue: A basic CFG can generate both structures, but it does not have enough semantic knowledge to automatically determine which interpretation is intended.

b) Limitations of Basic CFG

1. Ambiguity

A CFG may generate multiple valid parse trees for the same sentence.

For example:

the damage to the car

can have different PP attachment structures.

2. Subject–Verb Agreement

A basic CFG does not naturally represent grammatical features such as singular and plural.

For example:

The customer reports the accident. ✓

The customers report the accident. ✓

The customer report the accident. ✗

A simple CFG may not properly distinguish these forms unless additional grammar rules are created.

3. Long Sentences

Insurance customers may provide lengthy descriptions containing many clauses.

Example:

"While I was travelling to work, another vehicle suddenly hit my car, damaging the bumper and headlights, and I reported the incident to the insurance company."

A basic parsing approach may need to consider many possible structures, increasing processing complexity.

4. Conversational and Incomplete Input

Real customers may enter:

Car damaged yesterday.

or:

Accident near Chennai, front bumper damaged.

These are incomplete or conversational rather than well-formed sentences, making traditional CFG parsing difficult.

5. Lack of Probability

Basic CFG tells us whether a structure is grammatically possible, but it does not naturally tell us **which possible interpretation is more likely**.

c) Improved Parsing Approach

The improved system combines:

- **PCFG**
- **Feature Structures**
- **Earley Parsing**

Proposed Architecture

Customer Claim

↓

Preprocessing

↓

Tokenization

↓

Earley Parser

↓

Possible Parse Structures

↓

Feature Structure Checking

↓

PCFG Probability Ranking

↓

Best Parse

↓

Claim Information Extraction

↓

Insurance Database

1. PCFG

Probabilistic Context-Free Grammar (PCFG) assigns probabilities to grammar rules.

For example:

VP → V NP PP 0.6

VP → V NP 0.4

If multiple valid interpretations are generated, the system can compare their probabilities and select the **most probable parse**.

So:

CFG generates possible structures, while PCFG ranks those structures.

2. Feature Structure

Feature structures add grammatical information such as:

Number = Singular / Plural

Person = 1st / 2nd / 3rd

Tense = Past / Present / Future

For example:

The customer reports the damage.

The system identifies:

customer → Number = Singular

reports → Number = Singular

This helps enforce **subject–verb agreement**.

3. Earley Parsing

Earley parsing is suitable for ambiguous and complex sentences.

It works using three major operations:

Prediction → Scanning → Completion

It can maintain several possible interpretations while processing the sentence.

For example:

I → reported → the → damage → to → the → car

The parser can maintain different possible attachment structures rather than repeatedly restarting the

parsing process.

It is also useful when the input is **incomplete**, such as:

My car was damaged...

Claim Information Extraction

After obtaining the best parse, the system extracts:

Information	**Example**
Claimant	I
Damaged Object	Car
Cause/Event	Accident
Damage	Damage to car
Time	After the accident
Claim Amount	If provided

This information can then be stored in the insurance company's database.

d) Benefits of the Proposed Architecture

Accuracy

PCFG helps select the most probable interpretation, while feature structures improve grammatical consistency.

Ambiguity Resolution

Instead of simply producing multiple parses:

Parse 1

Parse 2

Parse 3

PCFG can rank them:

Parse 1 → 0.70

Parse 2 → 0.20

Parse 3 → 0.10

and select the most likely interpretation.

Scalability

The grammar can be expanded with insurance-specific concepts such as:

Vehicle

Accident

Damage

Claimant

Amount

Location

Date

Policy

Reliability

Combining syntactic parsing, grammatical features, and probability produces more reliable claim information than using a basic CFG alone.

2. Intelligent Railway Ticket Booking Assistant

Given passenger request:

"Book two tickets from Chennai to Delhi for my parents with AC sleeper."

a) Possible Syntactic Interpretations Using CFG

A simplified CFG could be:

S → VP

VP → V NP PP

NP → Num N

PP → P NP

NP → NP PP

There can be different attachments for:

"with AC sleeper"

Interpretation 1 – "with AC sleeper" modifies the tickets

Book [two tickets [from Chennai to Delhi] [for my parents] [with AC sleeper]].

Meaning:

Book two tickets from Chennai to Delhi for the parents, with **AC Sleeper as the class**.

This is the intended interpretation.

Interpretation 2 – "with AC sleeper" modifies the passengers

Book two tickets [for my parents [with AC sleeper]].

This could incorrectly suggest that **"with AC sleeper"** describes the passengers rather than the ticket/class.

Therefore, the PP attachment is ambiguous.

b) Problems with Top-Down Parsing

Top-down parsing starts from the start symbol and attempts to generate the input.

For example:

S

↓

VP

↓

V NP PP

↓

...

When the parser encounters ambiguity, it may try one possibility and later discover that it does not match the input.

It then **backtracks** and tries another possibility.

Example

with AC sleeper

↓

Try → Passenger attachment

↓

Doesn't fit

↓

Backtrack

↓

Try → Ticket attachment

↓

Correct

For long conversational queries with many possible attachments, repeated backtracking can become expensive.

Other problems include:

- Ambiguous phrase attachment
- Incomplete requests
- Long-distance dependencies
- Large search space
- Increased response time

This is particularly undesirable for a **real-time booking assistant**.

c) How Earley Parsing Helps

Earley parsing is designed to handle **general CFGs**, including ambiguous grammars.

Instead of repeatedly exploring the same possibilities, Earley parsing stores partial parsing information in its chart.

Its three main operations are:

Prediction

Predicts possible grammar structures.

Scanning

Matches the predicted structure with the actual input words.

Completion

Completes a constituent when its expected components have been recognized.

For:

"Book two tickets from Chennai to Delhi for my parents with AC sleeper."

Earley parsing can maintain alternative interpretations of:

with AC sleeper

and identify the possible attachment without repeatedly rebuilding the entire parse.

Incomplete Input

It can also maintain partial structures for inputs such as:

"Book two tickets from Chennai…"

This is useful in conversational systems because users often type or speak incomplete requests.

d) Comparison

Feature	Top-Down Parsing	CFG-Based Parsing	Earley Parsing
Approach	Starts from start symbol	Uses CFG grammar rules	Chart-based CFG parsing
Ambiguity	Can require backtracking	Can produce multiple parses	Handles ambiguity efficiently
Backtracking	Often significant	Depends on parser	Avoids repeated work using chart
Incomplete Input	Difficult	Depends on implementation	Better suited
Long Sentences	Can become expensive	Depends on parsing algorithm	Handles general CFG structures
Partial Parsing	Limited	Parser-dependent	Good support
Real-Time Suitability	Can suffer from delays	Depends on implementation	Better for complex queries
Main Advantage	Simple to understand	Flexible grammar representation	Handles ambiguity and partial input

Important distinction

CFG is a grammar formalism, not itself a parsing algorithm.

So in your comparison, it is better to say:

"A conventional CFG-based parser"

rather than treating CFG and Top-Down Parsing as exactly the same thing.

3. AI-Based Legal Contract Analysis System

a) Syntactic Structure Using CFG

Sentence:

"The supplier who delivered the equipment to the company must replace the defective components within thirty days."

A simplified CFG can represent it as:

S → NP VP

NP → Det N RC

RC → Pronoun VP

VP → V NP PP

VP → Modal VP

NP → Det N

PP → P NP

The basic structure is:

S

├── NP

│ ├── The supplier

│ └── Relative Clause

│ └── who delivered the equipment to the company

└── VP

├── must

└── replace the defective components

└── within thirty days

Main ambiguity

The phrase:

"to the company"

can potentially be attached to **"delivered"** or interpreted as part of another phrase.

The correct interpretation is:

The supplier delivered the equipment to the company.

The **supplier** is the entity performing the action **delivered**.

The system may incorrectly associate **company** with the delivery action as the responsible entity because

of the nested relative clause and PP attachment.

b) Limitations of Basic CFG

Basic CFG has difficulty because:

1. **Relative clauses** create nested structures.
2. **Prepositional phrases** can have multiple possible attachments.
3. Long legal sentences contain many nested phrases.
4. CFG does not naturally use semantic context.
5. It does not assign probabilities to competing interpretations.
6. Basic CFG does not automatically enforce grammatical features such as number, tense, or agreement.
7. Complex structures can produce many possible parse trees.

Therefore, simply using CFG may result in **incorrect identification of parties and obligations**.

c) Improved NLP Architecture

The proposed architecture combines:

CFG + Feature Structures + Earley Parsing + PCFG

Legal Contract

↓

Text Preprocessing

↓

Tokenization

↓

CFG Grammar

↓

Earley Parser

↓

Possible Parse Structures

↓

Feature Structure Validation

↓

PCFG Probability Ranking

↓

Best Syntactic Interpretation

↓

Semantic Information Extraction

↓

Contract Knowledge Base

Role of each technique

CFG: Defines the basic grammatical structure.

Earley Parsing: Efficiently handles long, nested, and ambiguous structures.

Feature Structures: Maintain information such as number, tense, person, and grammatical relationships.

PCFG: Assigns probabilities to competing interpretations and selects the most likely structure.

d) Step-by-Step Information Extraction

For:

"The supplier who delivered the equipment to the company must replace the defective components within thirty days."

Step 1 – Identify the parties

Supplier → Party responsible

Company → Receiving organization

Step 2 – Identify the relative clause

who delivered the equipment to the company

The parser identifies:

Subject → Supplier

Action → Delivered

Object → Equipment

Recipient → Company

Step 3 – Identify the main obligation

The main clause is:

The supplier must replace the defective components.

Therefore:

Responsible Party → Supplier

Obligation → Replace defective components

Step 4 – Identify the deadline

within thirty days

is extracted as:

Deadline → 30 days

Final extracted information

Field	**Extracted Information**
Supplier	The supplier

Field	Extracted Information
Action	Delivered / Replace
Equipment	The equipment
Company	The company
Obligation	Replace defective components
Deadline	Within 30 days

Why the improved approach is better

It separates the **syntactic analysis** from **semantic interpretation**, reducing the possibility of incorrectly identifying the company as the party responsible for delivery.

4. AI-Based E-Commerce Product Search System

a) Syntactic Ambiguity Using CFG

Query:

"Find laptops with a touchscreen for students under ₹60,000."

The intended interpretation is:

Find **laptops** that have a **touchscreen**, are suitable **for students**, and cost **under ₹60,000**.

A simplified structure is:

S
└── VP
  ├── Find
  └── NP
    ├── laptops
    ├── with a touchscreen
    ├── for students
    └── under ₹60,000

Ambiguity

The phrase:

"for students"

could potentially be attached to:

touchscreen

or:

laptops

The intended interpretation is:

Laptops
├── touchscreen

├── suitable for students

└── price < ₹60,000

b) Limitations of CFG

A basic CFG may have difficulty with:

- Ambiguous PP attachment
- Informal queries
- Missing words
- Elliptical queries
- Long product descriptions
- Multiple attributes
- Coordination
- Real-time processing

For example:

"Laptop under 60k touchscreen student."

This is understandable to a human but is not a complete grammatical sentence.

c) Improved Parsing Architecture

User Query

↓

Preprocessing

↓

Tokenization

↓

Earley Parser

↓

Possible Parse Structures

↓

Feature Structures

↓

PCFG Ranking

↓

Information Extraction

↓

Product Search Engine

↓

Ranked Products

Extracted features

The system can convert the query into structured constraints:

Product = Laptop

Feature = Touchscreen

Target = Students

Price = < ₹60,000

Role of the techniques

Earley Parsing → Handles incomplete and ambiguous queries.

Feature Structures → Represent product attributes and constraints.

PCFG → Selects the most likely interpretation.

CFG → Defines the grammatical patterns of queries.

d) Benefits

Search Precision

The system correctly separates:

Product → Laptop

Feature → Touchscreen

User Requirement → Students

Price Constraint → Under ₹60,000

This produces more relevant search results.

Response Time

Efficient parsing reduces unnecessary interpretation and allows the search engine to process natural-language queries quickly.

Customer Experience

Customers can type natural requests instead of manually selecting multiple filters.

For example:

"Find a gaming laptop with 16 GB RAM under ₹80,000."

The system can automatically extract the requirements.

5. Intelligent Airline Voice Assistant

a) Syntactic Structures and Ambiguity

Command:

"Change my flight to Mumbai tomorrow with two passengers."

Possible interpretation 1 — intended:

Change my flight

├── Destination → Mumbai

├── Date → Tomorrow

└── Passengers → Two

Possible interpretation 2:

Destination → Mumbai with two passengers

The phrase:

"with two passengers"

can therefore create an attachment ambiguity.

The intended meaning is that **two passengers are travelling**, not that Mumbai has an attribute involving two passengers.

b) Problems with Top-Down Parsing

A conventional top-down parser starts from the start symbol and tries to generate the sentence.

When it chooses the wrong interpretation, it may need to **backtrack**.

This can cause:

- Increased processing time
- Excessive backtracking
- Difficulty with ambiguous phrases
- Problems with incomplete commands
- Problems caused by speech-recognition errors

Correction example

Passenger says:

"Change my flight to Mumbai tomorrow... actually, make that Bangalore."

The system must understand that:

Mumbai

was replaced by:

Bangalore

A simple parser may have difficulty updating the previous interpretation.

c) Improved Architecture

Voice Input

↓

Speech Recognition

↓

Text Preprocessing

↓

Earley Parser

↓

Partial / Possible Parses

↓

Feature Structures

↓

PCFG Ranking

↓

Intent & Entity Extraction

↓

Booking Request

↓

Flight Booking System

↓

Confirmation

Feature Structure

The spoken request can be represented as:

Intent = Change Flight

Destination = Mumbai

Date = Tomorrow

Passengers = 2

After correction:

Intent = Change Flight

Destination = Bangalore

Date = Tomorrow

Passengers = 2

The system updates only the corrected field.

d) Processing Workflow

Step 1 – Speech Input

Passenger speaks:

"Change my flight to Mumbai tomorrow with two passengers."

Step 2 – Speech Recognition

Speech is converted into text.

Step 3 – Earley Parsing

The system identifies possible grammatical structures and handles incomplete input.

Step 4 – Feature Structures

Important information is represented as structured features:

Destination = Mumbai

Date = Tomorrow

Passengers = 2

Step 5 – PCFG

If multiple interpretations are possible, PCFG ranks them and selects the most probable interpretation.

Step 6 – Correction Handling

Passenger says:

"Actually, make that Bangalore."

The system updates:

Mumbai → Bangalore

instead of creating a completely new booking request.

Step 7 – Booking System

The final structured request is sent to the airline booking system.

Change Flight

Destination: Bangalore

Date: Tomorrow

Passengers: 2

Why this approach is suitable

The combination of **Earley Parsing + PCFG + Feature Structures** provides:

- Better ambiguity handling
- Support for partial input
- Reduced unnecessary backtracking
- Better handling of corrections
- Grammatical consistency
- Faster and more reliable conversational interaction